

TensorDex: A Compact, Tensor-Centric Storage System for Modern AI Models

Tingfeng Lan
University of Virginia
Charlottesville, VA, USA
erc8gx@virginia.edu

Zirui Wang
University of Virginia
Charlottesville, VA, USA
eeb9sd@virginia.edu

Yunjia Zheng
Harvard University
Cambridge, MA, USA
yunjia_zheng@g.harvard.edu

Zhaoyuan Su
University of Virginia
Charlottesville, VA, USA
acf7ea@virginia.edu

Juncheng Yang
Harvard University
Cambridge, MA, USA
juncheng@seas.harvard.edu

Yue Cheng
University of Virginia
Charlottesville, VA, USA
mrz7dp@virginia.edu

Abstract

Modern model hubs store hundreds of petabytes of large language models (LLMs), with fine-tuned variants dominating the storage footprint. These variants contain substantial cross-model redundancy that delta compression can exploit by storing only the difference between a target and a reference model. However, compression effectiveness depends critically on choosing a similar reference. At model-hub scale, this is challenging because model lineage metadata is often missing or unreliable, and different tensors within the same model may be most similar to tensors from different models. Consequently, model-level pairing leaves substantial redundancy unexploited.

We present TensorDex, a lossless, tensor-centric storage system that performs delta compression at tensor granularity. TensorDex decomposes models into tensors, uses compact bit-level fingerprints to predict tensor-pair compressibility, and incrementally organizes tensors into multi-center clusters to select effective bases as new models arrive. Evaluated on 2,890 randomly sampled Hugging Face models, TensorDex reduces storage footprint by 70.5%, 37% lower than the state-of-the-art design. It achieves 22.9 GB/s compression and 28.4 GB/s decompression throughput, 3.86× and 1.49× faster than the next-best system, respectively.

Keywords: model storage, tensor-centric storage, delta compression, large language models, model hubs

ACM Reference Format:

Tingfeng Lan, Zirui Wang, Yunjia Zheng, Zhaoyuan Su, Juncheng Yang, and Yue Cheng. 2026. TensorDex: A Compact, Tensor-Centric Storage System for Modern AI Models. In *Symposium on Operating Systems Principles (SOSP '26)*, September 29–October 2, 2026, Prague, Czech Republic.



This work is licensed under a Creative Commons Attribution 4.0 International License.

SOSP '26, Prague, Czech Republic

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2585-2/26/09

<https://doi.org/10.1145/3830418.3843872>

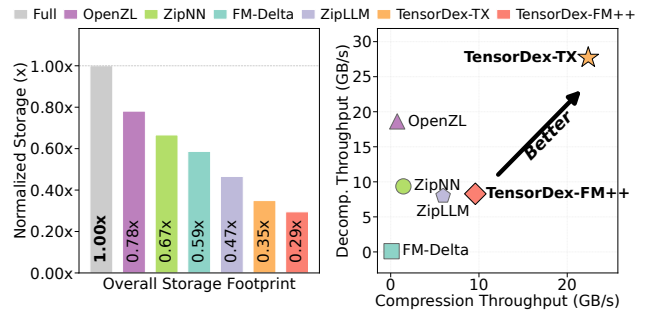


Figure 1. Left: Normalized storage relative to uncompressed (Full = 1.00× 40.11TB of randomly sampled Hugging Face models). TensorDex reduces total storage cost by 3.39×, achieving substantially lower storage footprint than state-of-the-art baselines. **Right:** TensorDex achieves high compression and decompression throughput.

Czech Republic. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3830418.3843872>

1 Introduction

Large language models (LLMs) have become foundational infrastructure across modern AI applications [49, 65, 67–69]. As open-source development accelerates, model hubs such as Hugging Face [30] now host millions of LLMs and are growing at an exponential rate [94]. The cumulative storage footprint of Hugging Face—models and datasets together—has surged from 0.28 PB in 2023 to over 76 PB by the end of 2025 [44]. Hugging Face reports that total storage doubled over the following five months and will cross 1 EB by the end of 2026 [16]; within that total, model storage grows faster than dataset storage [96], so model storage alone is set to reach exabyte scale around 2027.

A defining characteristic of this growth is that fine-tuned derivatives [12, 53, 83]—not base models—dominate both the model count and total storage footprint: by 2025, fine-tuned models account for 99.1% of total storage and 99.6% of all models (Fig. 2). These derivatives proliferate rapidly as users specialize models for diverse downstream tasks, producing highly heterogeneous collections of closely related but structurally divergent variants. As a result, storage growth is

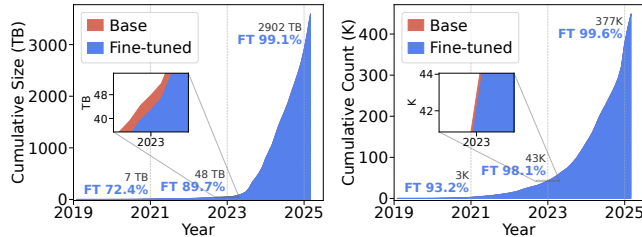


Figure 2. Cumulative storage size (left) and model count (right) on Hugging Face from 2019 to 2025. Fine-tuned models (blue) dominate both metrics, accounting for 99.1% of storage and 99.6% of model count by 2025, while base models (red) remain a small fraction.

driven not by a few large base models, but by vast families of fine-tuned models that differ subtly yet occupy full model size. This trend is further amplified by model development practices: large-scale training pipelines often retain numerous intermediate checkpoints, each storing full model weights and the optimizer state. For example, recent reports indicate that production training workflows may persist all checkpoints for traceability [92], significantly compounding the overall storage burden.

Delta compression (i.e., storing only the difference between a fine-tuned model and a reference base) is a natural solution to reduce storage footprint. Recent works have demonstrated that it significantly outperform direct compression of model weights [74, 94]. However, deploying delta compression at real-world scale exposes three fundamental limitations of existing approaches:

Missing Lineage Metadata. ZipLLM [94] relies on Hugging Face model-card metadata to determine model lineage for delta compression. However, model cards are optional. Our analysis of over 2.7M Hugging Face models on March 2026 shows that only 25.8% contain usable base-model metadata. When metadata is unavailable, ZipLLM falls back to brute-force pairwise bit distance—computing the exact Hamming distance over model pairs—to infer model similarity. This requires reading every byte of every candidate pair, incurring I/O cost that grows *quadratically* with the size of the model hub, making it impractical for real-world deployment.

Model Pair Sensitivity. Even with accurate lineage, assigning all tensors in a fine-tuned model to a single base is fundamentally suboptimal. Fine-tuning induces continuous parameter drift, causing tensors to move progressively farther from any fixed base model. This drift is amplified by the proliferation of specialized variants: models fine-tuned for reasoning, safety, coding, alignment, or multilingual tasks exhibit significantly different parameter shifts [21, 36, 48]. As a result, compression effectiveness becomes highly sensitive to the choice of base: closely related pairs yield low-entropy deltas and high reduction, while pairing based on a fixed pre-trained base or user-provided metadata often leads to significantly lower compression.

Compression Granularity. Model storage redundancy exists at the fine-grained tensor level, not the model level.

Different tensors within the same model (e.g., embeddings, attention projections, MLP layers) follow distinct training dynamics and thus exhibit different similarity relationships across derivatives. For many tensors, the best bases for delta compression [94, 103] do not come from the model’s declared base, or even from the same variant; instead, they are scattered across different models. This mismatch imposes an inherent upper bound on coarse-grained model-level delta compression: forcing all tensors to share a single base leads to systematically suboptimal pairings, limiting achievable reduction ratios even under ideal conditions. Model-level compression therefore obscures substantial cross-model redundancy—a coarse, single-base assignment fails to exploit the rich, tensor-level similarity structure present in real-world large-scale model storage.

To this end, we present TensorDex, a first-of-its-kind, loss-less, *tensor-centric* model storage system that departs from model-level designs and instead *uncovers and exploits hidden cross-tensor relationships across models*, treating tensors as the *first-class citizen* for delta compression.

TensorDex has four components. First, TensorSketch is a compression-aware fingerprint that captures bit-level structure and enables efficient similarity estimation without accessing full tensor contents. Second, a lightweight regressor that predicts the compressibility of tensor pairs using TensorSketch without reading full tensor data or performing actual compression. Third, FlexSplit, a scalable clustering algorithm that identifies the best tensor bases incrementally and adaptively as new models are uploaded. Fourth, an optimized codec that performs high-performance compression and decompression. Together, these components enable scalable compression planning that adapts to the heterogeneous and evolving structure of real-world model hubs.

This paper makes the following contributions:

- This is, to our knowledge, the first work that shows LLM storage redundancy fundamentally emerges at the tensor level, not the model level nor traditional chunk level.
- We introduce a *novel fingerprint metric*, *TensorSketch*, to captures both the numerical distribution and coarse structural layout of each tensor, enabling scalable and efficient similarity estimation.
- We design a regression model to predict model compressibility based on TensorSketch without reading the full tensor or performing compression.
- We design a scalable, incremental clustering algorithm that enable continuous model ingestion by discovering multi-center tensor clusters and selecting high-quality base-delta pairs.
- We implement and optimize two compression codecs that enable high-performance and compression ratio. Together, these components form TensorDex.
- Evaluated on 2,890 randomly sampled models from Hugging Face, TensorDex reduces storage footprint by 70.5%, 37% lower than state-of-the-art design (ZipLLM (Fig. 1)).

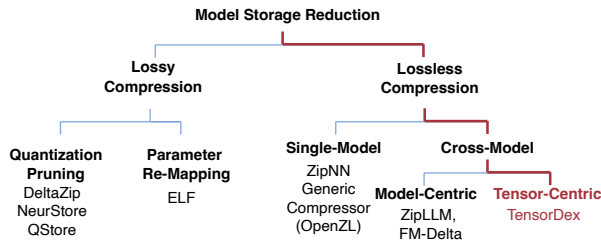


Figure 3. The design scope of TensorDex.

Meanwhile, it achieves 22.9 GB/s compression and 28.4 GB/s decompression throughput, 3.86× and 1.49× faster than the next best system, respectively.

Open-source release. TensorDex’s source code is publicly available at <https://github.com/ds2-lab/TensorDex>.

2 Background and Related Work

This section reviews existing storage reduction techniques. A high-level comparison of these methods is summarized in Fig. 3.

2.1 Traditional Storage Reduction

General-purpose Compression. General-purpose compressors such as Zstandard [19] and Brotli [1] exploit local byte-level redundancy via dictionary-based [95, 106] and entropy coding [26, 29, 41, 64, 76, 90], with further gains when the data type is known [13, 14, 52, 54, 59, 61, 82, 88, 91]; none of them exploits redundancy *across* models. Lossy scientific codecs [27, 28, 56, 58, 86] and inference-time quantization [25, 31, 57, 99, 104, 105] cannot guarantee the exact recovery that model hubs require.

Data Deduplication. Deduplication retains only unique blocks and replaces duplicates with pointers. File-level hashing, as in Git LFS [35], treats a file as atomic, so one changed byte hides all content it still shares with earlier versions. Content-Defined Chunking (CDC) [71] recovers that redundancy below file granularity using rolling hashes, and is adopted in NetApp ONTAP [72, 73], Dell EMC Data Domain [24], FastCDC [97], and TiDedup [75]. ZipLLM [94] instead content-hashes each *tensor* and stores it once, capturing exact duplicates across fine-tuned variants that share identical layers, without the metadata overhead of variable-sized chunking.

2.2 Model-aware Storage Reduction

Single-model Compression. Intra-model compression reduces the storage footprint of individual models by exploiting redundancy in their floating-point parameters. ELF [84] eliminates exponent bits by mapping weights into a normalized range, but it is inherently lossy and unsuitable for model hubs that require exact recovery. ZipNN [39] improves compressibility by reordering float bytes to isolate compressible fields like sign and exponent bits, enabling more effective

entropy coding. However, these methods only capture local, single-tensor redundancy and cannot exploit shared patterns across models.

Cross-model Delta Compression. Delta compression reduces storage by encoding a model’s weights relative to a chosen *base* model. The element-wise difference—by subtraction for floating-point data or XOR for byte-level representations—yields a *delta* of much lower entropy, which a general-purpose compressor then stores. FM-Delta [74] compresses arithmetic residuals between base models and their variants, but offers limited throughput and no pairing guidance, making it hard to scale to hubs with millions of models. ZipLLM [94] applies bitwise XOR delta compression to scale storage optimization across large model hubs, but depends on `base_model` metadata and pairs at model granularity (§1). DeltaZip [103] further prunes the delta matrix with quantization. NeurStore [98] and QStore [79] explore cross-model redundancy via lossy quantization and approximate sharing. In contrast, TensorDex preserves exact recovery and targets fine-grained tensor-level redundancy through lossless compression.

2.3 Vector Similarity Search

A key in delta compression is to find the most similar pair to compress, which shares similarity with vector search. High-dimensional vector search relies on Approximate Nearest Neighbor (ANN) algorithms to address the scaling limits of exact retrieval, including hashing-based [5, 22, 33, 80], graph-based [32, 60, 63, 93], tree-based [11, 81], and quantization-based [6, 51] methods.

Graph-based indices currently offer the best balance between speed and accuracy: Hierarchical Navigable Small World (HNSW) [62] routes greedily through a hierarchy of proximity graphs, achieving logarithmic search complexity. Vector search is often designed for small vectors, e.g., up to 3,072 dimensions; however, tensors can have millions of elements (dimensions). Directly using existing vector search algorithms will be computationally prohibitive.

A separate line of work keeps such indexes serviceable under continuous updates rather than rebuilding them: SPFresh [100] splits partitions in place, triggered by partition-size balance, and Quake [70] re-partitions under a query-latency cost model over partition sizes and access frequencies.

3 Problem Formulation and Challenges

3.1 Empirical Insights into LLM Delta Compression

While prior works [74, 94] demonstrate that delta compression can be highly effective in reducing storage footprint, real-world model hubs exhibit more complex and heterogeneous structures than those in controlled experiments.

Observation #1 (missing lineage): Model lineage information is often missing or unreliable in real-world model repositories. Existing systems assume model family information is available (e.g., via model cards [42]); however,

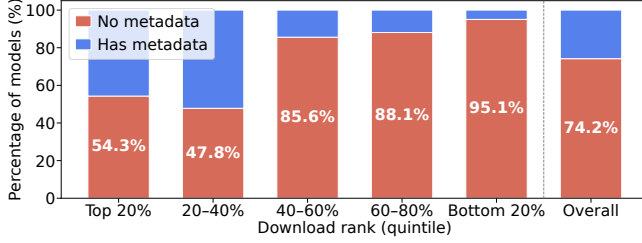


Figure 4. Distribution of Hugging Face model lineage metadata by download rank. Most models (74.2% overall) lack this information. such information is often missing or unreliable on the Hugging Face model hub. As shown in Fig. 4, we analyzed all 2,751,212 Hugging Face model repositories as of March 2026 and found that only 25.8% are tagged with model-family information. Even the top 20% hot models by download count already have a 54.3% missing rate for lineage metadata. For cold models below the 40th percentile in downloads, which make up the majority of the hub, the missing rate consistently exceeds 88%. Among those, the metadata is often incomplete or unreliable, as users frequently copy metadata from other repositories or manually modify fields in ways that break the true lineage [23, 55, 85].

Implications

It is nearly impossible to rely on model-family lineage as a reliable signal for delta compression in real-world model hubs due to missing and unreliable metadata.

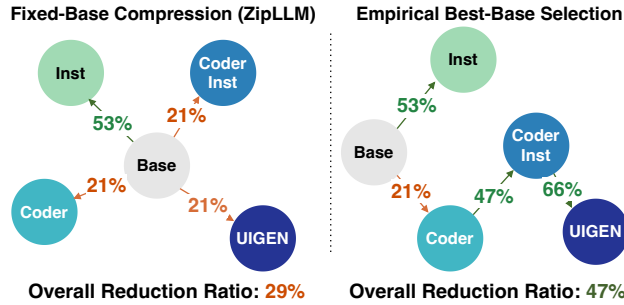


Figure 5. Choosing the delta-compression base is crucial. Both panels show five Qwen2.5-3B variants. Edge labels denote the data-reduction ratio from delta-compressing each variant to its assigned base. **Left:** Fixed-base compression pairs each variant with the pre-trained *base*, achieving only 29% overall reduction. **Right:** Selecting the empirical best base per model raises overall reduction to 47%.

Observation #2 (pair sensitivity): Delta compression is highly sensitive to pair selection. Across LLM repositories, we find that compression effectiveness is governed by the similarity between the target and its base: closely related models yield low-entropy deltas and high reduction, while pairing based on a fixed pre-trained base or user-provided metadata often leads to substantially lower reduction (Fig. 5). Consequently, pair selection becomes a first-order factor in

determining storage efficiency, and naive pairing strategies can leave significant compression potential unrealized.

Implications

Using pre-trained model as the base is insufficient; effective delta compression requires data-driven pairing that reflects true model weight similarity.

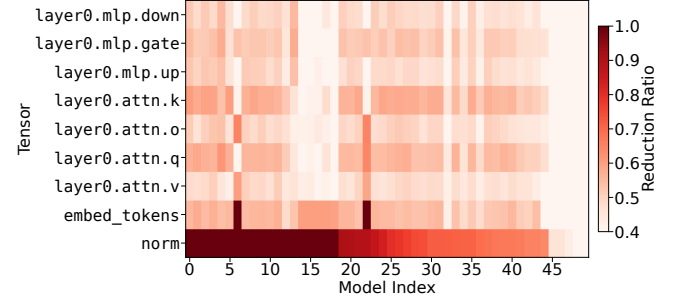


Figure 6. Different tensors within a single model exhibit heterogeneous storage-reduction ratios and the best bases vary. Each cell shows the reduction ratio when compressing a tensor (row) against the corresponding tensor in a candidate model (column).

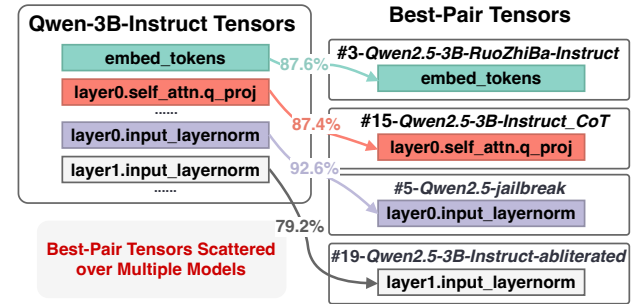


Figure 7. Optimal compression pairing emerges at the tensor level. The best bases for different tensors in the same model often reside in a *different* model, rather than in a single shared base.

Observation #3 (compression granularity): The best compression unit is fine-grained—at the tensor level, not the model level. While prior systems typically apply delta encoding at the whole-model granularity [74, 94], our analysis shows that the most effective compression pairing often emerges at the level of individual tensors. Within a single model, different tensors exhibit distinct training dynamics: attention layers, MLP blocks, and embedding matrices drift in different ways as they adapt to data and task objectives. Fig. 6 shows that the tensors from a single model may exhibit different data reduction ratios when compressed against a chosen base model (each column). Meanwhile, the best model for each tensor is different (darkest in each row). Therefore, no single base model consistently serves as the best base for all tensors of a target model. Instead, the optimal

pairing varies tensor by tensor, revealing that fine-grained tensor-level decisions are essential for maximizing delta compressibility. Fig. 7 shows that the best bases of four random tensors from Qwen-3B-Instruct come from four entirely different models.

Implications

Model storage systems should apply delta compression at the tensor level to better capture cross-model redundancy. Choosing pairs per tensor, rather than per model, improves the data reduction ratio.

3.2 Problem Formulation

Problem Definition. Minimizing model storage requires jointly choosing which tensors act as bases and which are stored as compressed deltas, a combinatorial optimization shaped by pairwise compressibility estimates that are themselves costly to compute. We formulate this problem below and highlight three key challenges.

Optimization Objective. Storing a tensor as a base incurs its full size; storing it as a compressed delta reduces cost, but the reduction depends on the specific base it references. Formally, the optimization problem is:

$$\min_{\text{base selection}} \left(\sum \text{cost}_{\text{base}} + \sum \text{cost}_{\text{delta}} \right).$$

pairing decisions → min base selection base storage cost comp. delta storage cost

3.3 Optimization Challenges

The optimization objective is an instance of the *facility-location problem* [4, 8], which is NP-hard and typically requires heavy integer linear programming (ILP) solvers [9, 37, 45]. Solving it at scale is challenging for three reasons:

Challenge #1 (C1): Estimating pairwise tensor similarity is expensive. Determining whether tensor t_i is similar to tensor t_j requires loading both tensors in full—each ranging from millions to billions of parameters—making brute-force pairwise comparisons prohibitively costly at the model-hub scale. A natural alternative is approximate nearest-neighbor (ANN) search [22, 32, 63, 93], but it suffers from the well-known *curse of dimensionality* [10, 46]: as dimensionality grows into the millions, distance concentration renders nearest-neighbor queries no more informative than random selection, and the memory overhead of indexing billion-scale tensors as high-dimensional vectors is itself prohibitive.

Challenge #2 (C2): Similarity does not imply compressibility. Even with an efficient similarity measure, a fundamental gap remains: standard distance metrics (e.g., L_2 , cosine) operate at the value level and do not capture *bit-level* patterns that govern delta compressibility. Two tensors that

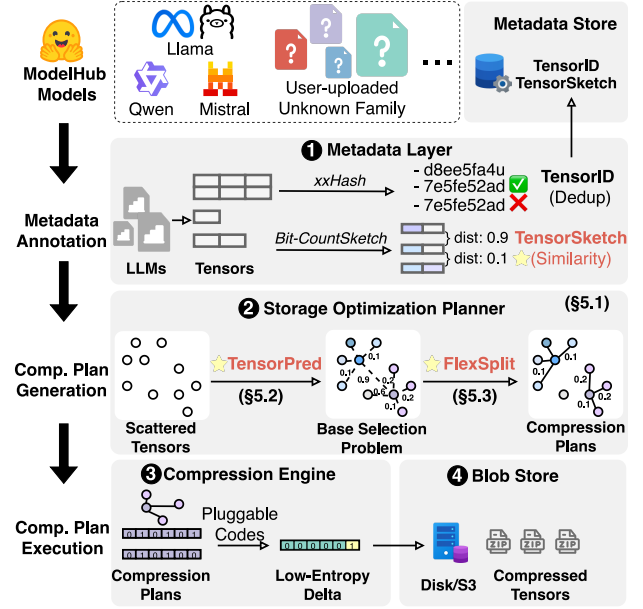


Figure 8. TensorDex architecture and workflow.

are close in L_2 space may still produce poorly compressible deltas because their byte-level entropy patterns differ.

Challenge #3 (C3): Selecting bases is combinatorial and dynamic. Even with accurate pairwise compressibility estimates, choosing which tensors to serve as bases remains NP-hard [4, 8]. For example, a projection tensor in LLaMA2-7B can choose from over 120,000 possible base tensors. Exact solvers such as ILP [37] are computationally intractable beyond a few hundred tensors. Classical approximation algorithms, including primal-dual [47] and local-search methods [7], offer theoretical guarantees but still require materializing the full pairwise cost matrix and are designed for static inputs.

Moreover, the tensors stored in real-world model hubs are dynamic: as new models arrive, previously selected bases may become suboptimal, triggering cascading reassignments that require recompressing and re-pairing of affected tensors.

Together, these challenges call for a system that can efficiently estimate pairwise similarity without full-tensor access (C1), predict compressibility from similarity (C2), and select bases at scale in a dynamic, incremental fashion (C3).

4 TensorDex Design

4.1 System Overview

Architecture. TensorDex treats tensors as first-class entities and explicitly reasons about cross-tensor relationships across a model hub, pairing each tensor with a similar base and delta-compressing it to minimize storage footprint. As shown in Fig. 8, TensorDex comprises four components: ① a content-aware tensor metadata layer (TensorID and TensorSketch), ② a compression planner (TensorPred and FlexSplit), ③ a compression engine (TensorX and FM++), and ④ a storage

layer for tensor blobs on generic backends such as local disk or cloud object stores (e.g., S3 [2], R2 [17]).

Workflow. When a batch of models is uploaded, the metadata layer ① parses each model into individual tensors and assigns two types of metadata: a TensorID content hash for exact deduplication and a compact TensorSketch fingerprint for similarity estimation (C1). The compression planner ② then uses TensorPred to convert fingerprint distances into predicted storage reduction ratios (C2), and FlexSplit to derive a base-delta clustering plan (C3). The compression engine ③ materializes this plan by computing and compressing deltas, and writing blobs to the storage layer ④. When a model is requested, the storage layer and compression engine cooperate to fetch bases, apply deltas, and reconstruct full tensors on demand.

Content-aware Tensor Metadata. TensorID is a hash [18] computed over raw bytes to detect exact duplicates. TensorSketch is a compact CountSketch-based [15] fingerprint that captures the bit-level distribution of tensor data, enabling efficient estimation of pairwise normalized Hamming distances (§4.2).

Compression Planner. The planner determines which tensors should be stored as bases and which should reference others via deltas, thereby achieving a low storage footprint for both bases and deltas. It relies on two methods: TensorPred (§4.3), a lightweight regression model that converts TensorSketch distances into predicted reduction ratios; and FlexSplit (§4.4), an incremental clustering algorithm that uses these predictions to select bases and assign tensors.

Compression Engine. The compression engine carries out planner-generated compression plans by computing and compressing deltas (§5.2) to generate the final compressed blobs. The engine is fully decoupled from plan generation and can be executed in parallel.

Storage Layer. TensorID, TensorSketch, and model-tensor mappings are stored in a metadata database (PostgreSQL [77]) supporting fast deduplication checks and fingerprint-based similarity queries at ingestion time. Each tensor blob is stored in a safetensors-compatible format [43] extended with compression flags, unifying compressed and uncompressed tensors under the same format.

4.2 TensorSketch: Compression-aware Fingerprint

The goal of TensorSketch is to provide a compact fingerprint for each tensor that captures the bit-level numerical distribution and structural layout—the two aspects that jointly determine whether two tensors are good candidates for delta compression.

TensorSketch Design. Existing fingerprinting approaches fall short for our purpose. Standard CountSketch [15] operates on floating-point values and preserves *value distribution*, but delta compression works at the *bit level*: what matters is not numerical distance per se, but the fraction of bits that differ between two tensors, i.e., the normalized Hamming

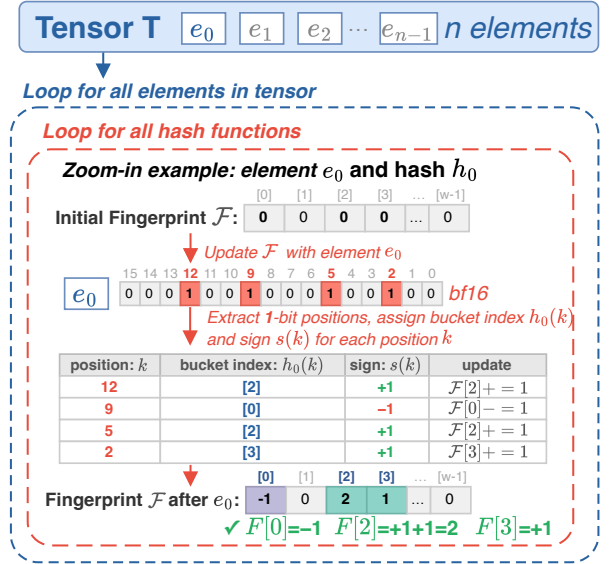


Figure 9. An example of TensorSketch fingerprinting.

distance [38] over their raw binary representations. A fingerprint that directly captures per-bit flip statistics is therefore a more faithful proxy for delta compressibility than one based on scalar magnitudes. We therefore introduce TensorSketch, a bit-level CountSketch construction that hashes each element’s individual bit positions into sketch buckets, capturing both bit distribution and layout in a compact, fixed-size fingerprint.

TensorSketch Construction. As shown in Fig. 9 and Alg. 1, TensorSketch addresses this by treating the tensor’s raw bytes as a flat bit sequence of $p * n$ bits, grouped into n elements of p bits each (e.g., $p=16$ for float16/bfloat16, $p=32$ for float32), and building a two-dimensional sketch table $\mathcal{F} \in \mathbb{R}^{d \times w}$ where d is the depth (number of independent hash rows) and w is the width (number of buckets per row). Crucially, each of the p bit positions in every element is treated as an independent binary signal: for bit position $k \in \{0, \dots, p-1\}$ of element i , a dedicated pair of hash functions (h_r, s_r)—seeded independently per row r —maps the element-bit-position pair (i, k) to a bucket and a sign. This *per-bit hashing* ensures that structural differences at any bit position (exponent, mantissa, or sign) are reflected in the sketch, making TensorSketch sensitive to the bit-level divergence that drives delta compression efficiency.

Hamming Distance Estimation. Given two fingerprints $\mathcal{F}^i$ and $\mathcal{F}^j$, the Hamming distance between the underlying tensors is estimated from the per-row squared ℓ_2 norm of the difference $\mathcal{F}_r^i - \mathcal{F}_r^j$. Taking the *median* over all d rows provides robustness against outlier hash collisions:

$$\hat{\mathcal{H}}(i, j) = \text{median}_{r=1}^d \left\| \mathcal{F}_r^i - \mathcal{F}_r^j \right\|_2^2. \quad (1)$$

aggregation for robustness

sketch fingerprint

Algorithm 1: TensorSketch Fingerprinting

Input: Params: sketch depth d ; sketch width w
 Data: Tensor t with $(e_0, \dots, e_{n-1})$, each p bits
 Hash functions: hash families $\{h_r\}_{r=1}^d$ with $h_r: [n] \times [p] \rightarrow [w]$ and $\{s_r\}_{r=1}^d$ with $s_r: [n] \times [p] \rightarrow \{-1, +1\}$
Output: TensorSketch $\mathcal{F} \in \mathbb{R}^{d \times w}$.

```

1 Initialize  $\mathcal{F}[0..d-1][1..w] \leftarrow 0$ ;
2 for  $i \leftarrow 0$  to  $n-1$  do
3   if  $e_i = 0$  then
4     skip; // all bits zero
5   end
6   for  $r \leftarrow 0$  to  $d-1$  do
7     for each bit position  $k$  where  $e_i[k] = 1$  do
8        $j \leftarrow h_r(i, k)$ ; // bucket index
9        $\sigma \leftarrow s_r(i, k)$ ; // sign
10       $\mathcal{F}[r][j] \leftarrow \mathcal{F}[r][j] + \sigma$ ;
11    end
12  end
13 end
14 return  $\mathcal{F}$ ;

```

The normalized Hamming distance, $\hat{p}(i, j) = \hat{H}(i, j) / (pn)$, lies in $[0, 1]$: near zero indicates near-identical bit patterns (highly compressible bit-level delta), while ≈ 0.5 indicates random divergence.

4.3 TensorPred: Reduction Ratio Prediction

TensorSketch estimates pairwise similarity, but the optimization formulation in §3 requires *quantitative reduction ratios* to evaluate each base-delta assignment (C2). TensorPred bridges this gap with a lightweight regression model that converts TensorSketch distances into predicted reduction ratios without materializing any actual delta.

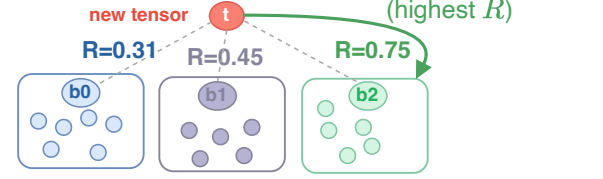
Key Observation. Under bit-level delta compression (detailed in §5.2), the fraction of differing bits between two tensors reflects the entropy of the resulting residuals and hence the achievable reduction ratio. The normalized Hamming distance $\hat{p}$ estimated by TensorSketch correlates with this fraction. However, the relationship between $\hat{p}$ and reduction ratio is non-linear: at small $\hat{p}$, a marginal increase in Hamming distance disproportionately raises the fraction of large-magnitude residuals, causing a sharp drop in reduction; at larger $\hat{p}$, residuals are already broadly distributed and additional divergence has diminishing effect—reduction ratio approaches 0 (no gain). Binary entropy $\mathcal{S}(\hat{p}) = -\hat{p} \log_2 \hat{p} - (1-\hat{p}) \log_2 (1-\hat{p})$, scaled as $\tau = 8\mathcal{S}(\hat{p})$, captures the per-element bit uncertainty a compressor must resolve and naturally models this non-linear regime change.

Model Formulation. Let $\hat{p}$ denote the normalized Hamming distance estimated by the TensorSketch distance estimator (§4.2). We define the following auxiliary feature:

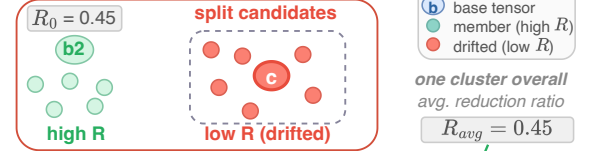
$$\tau = 8\mathcal{S}(\hat{p}), \quad (2)$$

Phase I - Greedy Assignment

Goal: $b^* = \operatorname{argmax}_{b \in B} R(t, b)$

**Phase II - New Center Splitting**

Before:



↓ Promote $c \rightarrow$ new base, reassign members

After: $\Delta R_{avg}(c) > 0 \rightarrow \text{commit split}$

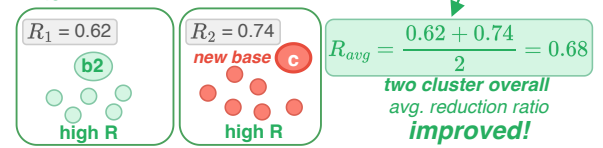


Figure 10. Example workflow of FlexSplit.

where $\mathcal{S}(\cdot)$ is the binary entropy function and the factor of 8 normalizes τ to the per-byte bit uncertainty. The predicted reduction ratio is:

$$\mathcal{R}(\hat{p}) = \underbrace{\alpha \hat{p}}_{\text{bit divergence}} + \underbrace{\beta \tau}_{\text{entropy cost}} + \underbrace{\gamma (\hat{p} \cdot \tau)}_{\text{nonlinear correction}} + \underbrace{\epsilon}_{\text{bias}}, \quad (3)$$

where $(\alpha, \beta, \gamma, \epsilon)$ are regression coefficients fitted offline on a corpus of real tensor pairs with their measured reduction ratios, and the result is clipped to $[0, 1]$. The three terms provide complementary signals: $\hat{p}$ captures raw bit divergence, τ captures its information-theoretic cost, and $\hat{p} \cdot \tau$ models the nonlinear coupling between divergence magnitude and entropic uncertainty (see Fig. 13).

4.4 FlexSplit: Adaptive Tensor Clustering and Cluster Splitting

Finding the best base is an NP-hard problem and ILP solvers are too slow for online systems. We design FlexSplit, a two-phase online algorithm that operates entirely on TensorSketch fingerprints and TensorPred estimates. First, it greedily assigns each arriving tensor to the cheapest existing base, then periodically splits overgrown clusters.

Phase I: Greedy Incremental Assignment. As new models are uploaded to the model hub, TensorDex processes each model's tensors in turn. The first tensor with unseen shape and name naturally seeds a new cluster as its initial base. For every subsequent tensor, FlexSplit determines which existing

base yields the highest predicted reduction ratio and assigns the tensor to that base’s cluster. As illustrated in Fig. 10 (top), a new tensor t is evaluated against three existing bases b_0 , b_1 , b_2 with predicted reduction ratios 0.31, 0.45, and 0.75, respectively; FlexSplit assigns t to b_2 because it offers the highest predicted reduction. Since all comparisons operate on compact TensorSketch fingerprints rather than raw tensor bytes, each assignment requires at most $O(k)$ distance evaluations over the current base set $\mathcal{B}$, where $k = |\mathcal{B}|$ is typically small. With approximate nearest-neighbor indexing, this cost further reduces to $O(\log k)$ per tensor.

Improvement Opportunities from New Arrivals. Greedy assignment connects each tensor to the best base *available at the time of ingestion*. As new models arrive, however, they introduce tensors that may serve as better centers for existing members. In other words, the new arriving tensors improve the solution space that the current static assignments do not reflect. Fig. 10 (Phase II, “Before”) illustrates this: the cluster around b_2 mixes well-matched members (green, high $\mathcal{R}$) with poorly-matched ones (red, low $\mathcal{R}$) that would compress better against a newly available center, dragging the cluster-wide average $\bar{\mathcal{R}}$ down to 0.45. To capture these improvement opportunities without rerunning global optimization, FlexSplit periodically revisits each cluster and asks whether promoting an existing member to a new base would reduce the projected storage cost.

Phase II: Multi-Center Splitting. Phase II revisits each cluster and asks whether promoting an existing member to a new base would improve the cluster-wide reduction ratio, defined as the total bytes saved by delta-encoding non-base tensors against their best base, divided by the total raw size of all members. Bases themselves are stored in full and contribute zero reduction. Initially each cluster has a single base, so the initial ratio serves as the baseline.

FlexSplit solves this greedily. Starting from the original base, it first identifies *split candidates*: tensors whose predicted reduction ratio falls significantly below the cluster average reduction ratio, indicating poor alignment with the current center. For each candidate, it evaluates the gain of promoting that tensor to a new base. Promoting a candidate trades off its own delta savings (now stored in full) against higher reduction ratios for nearby tensors that better align with it. A split is committed only when the split benefit is positive. FlexSplit selects the candidate with the largest gain, promotes it to a new base, and reassigns all members whose reduction ratio improves under this new center. As shown in Fig. 10 (bottom, “After”), promoting candidate c to a new base partitions the original cluster into two coherent sub-clusters—both now exhibit high $\mathcal{R}$ for all members, lifting the cluster-wide average from $\bar{\mathcal{R}} = 0.45$ to 0.68.

Splitting is *cluster-local*: the promoted base comes from within the cluster and only its members are re-scored. Unlike an incremental vector index, where a split must reassign boundary vectors in *neighboring* partitions to preserve recall [70, 100], the storage-cost objective decomposes per cluster, so no cascading refinement follows.

5 Implementation

TensorDex is implemented in roughly 12K lines of Python and 3K lines of optimized C/Rust, and integrates directly with Hugging Face safetensors model formats. This section describes the two main implementation aspects: the metadata engine that organizes the metadata of base and delta tensors (§5.1), and the SIMD/AVX2-accelerated operators that make ingestion and decompression fast (§5.2).

5.1 Metadata Engine

Metadata Schema. TensorDex maintains per-tensor metadata in PostgreSQL for distributed deployments. Each tensor metadata entry stores:

```
(model_id, tensor_name, tensor_id,
  tensor_sketch, dtype, shape)
```

As the original safetensors format already includes dtype and shape in its header, TensorDex extracts them at zero cost. These fields allow the planner to filter out structurally incompatible candidates before computing any fingerprint distances, reducing the search space for valid base-delta pairs.

TensorID is a 16 B xxHash128 [18] digest over the tensor’s raw bytes, used both as the deduplication key and as the content address that a decoded tensor is verified against at retrieval time. TensorSketch uses $d=2$ rows of $w=1024$ 32-bit counters, i.e., 8 KB per tensor (Table 2).

Search Index. To efficiently retrieve the most similar base for a given tensor, TensorDex builds an HNSW [62] index *over TensorSketch fingerprints*. At ingestion time, each new tensor’s sketch is inserted into the index; at query time, the planner performs an approximate nearest-neighbor lookup to identify candidate bases in $O(\log n)$ time, avoiding exhaustive pairwise comparisons across the full tensor pool.

Deletions. Deletion is deferred: TensorDex tombstones the tensor and reclaims its bytes in a later offline maintenance transaction. If the tensor is a base with live dependents, that transaction promotes another member of its cluster to base and recompresses the dependents against it, swapping the metadata pointer atomically once the new blob is written. The rewrite stays within that one cluster.

5.2 High-Performance Codec

All compute-intensive operations in the ingestion and retrieval paths are implemented as SIMD-accelerated kernels in C and Rust, enabling TensorDex to saturate memory bandwidth on modern hardware.

Table 1. Evaluation dataset. All reduction ratios are measured against the 40.11 TB size deduplication based on TensorID.

Models	Before dedup.		After dedup.	
	Tensors	Size	Tensors	Size
2,890	961,509	44.86 TB	741,689	40.11 TB

TensorID. Tensor deduplication hashes each tensor’s raw bytes using xxHash128 with AVX2 vectorization.

TensorSketch. The TensorSketch kernel (§4.2) is implemented in Rust as a fused single-pass streaming operator over zero-copy mmap buffers. Per-bit hash and sign lookups are SIMD-parallel, and sketch accumulation requires no intermediate buffers.

Pluggable Codec Engine. TensorDex’s compression engine exposes a pluggable codec interface, decoupling planning from compression so that any lossless compressor can be used. We implement two codecs targeting different tradeoffs:

- **Throughput-oriented TensorX** extends BitX [94] with chunk-level parallelism and byte-plane separation. Chunks are compressed independently in parallel; byte grouping (e.g., first byte of different BF16 floats compressed together) is used to exploit the near-zero skew of fine-tuned weight deltas for better entropy coding. All kernels are SIMD/AVX2-accelerated.
- **Reduction-oriented FM++** is a lossless codec for fine-tuned models, extended from FM-Delta [74]. We add native BF16 support and chunk-level parallel execution, improving throughput from ~100 MB/s to 9,821 MB/s (a ~98× speedup) while retaining its higher reduction ratio over TensorX.

6 Evaluation

6.1 Experimental Setup

Dataset. We evaluate TensorDex by reconstructing the random-sampled model trace used in ZipLLM [94]. The details are shown in Table 1. It spans a broad range of architectures: 968 models derived from Qwen2.5 [102], 151 from Qwen3 [101], 139 from Mistral [49], 114 from Llama-3 [65], 1,431 from Llama-3.1 [66], 47 from Llama-3.2 [68], 135 from Gemma-2 [78], and 63 from Gemma-3 [87]. The trace is sampled randomly *within* these representative fine-tuned families rather than across the entire hub. LoRA adapters [40] and quantized checkpoints are tensor sets as well and need no pipeline change beyond a shared dtype between delta and base, but full-weight fine-tunes and checkpoints dominate hub storage (99.1%, Fig. 2) and are our focus here.

TensorDex Configurations. TensorDex’s compression engine exposes a pluggable codec interface, allowing different delta compressors to be swapped without changing the planning pipeline. We evaluate two configurations:

- **TensorDex-TX** (FlexSplit + TensorX) for optimized compression/decompression throughput.

Table 2. Tensor metadata statistics and overhead.

Method	#Tensors	Per tensor	Total size
TensorID (xxHash128)	741,689	16 B	11.97 MB
TensorSketch ($d=2$, $w=1024$)	741,689	8 KB	6.08 GB
Metadata overhead		0.015%	

- **TensorDex-FM++** (FlexSplit + FM++) for optimized reduction ratio.

Compression Planner Baselines. To evaluate the end-to-end storage reduction achieved by TensorDex’s planning pipeline (TensorPred + FlexSplit), we compare against production-grade systems:

- **ZipLLM [94]:** the state-of-the-art model-aware lossless storage reduction system. It clusters models using either (i) the `base_model` field from Hugging Face model cards (when available) or (ii) a bit-distance metric [94] that measures the number of differing bits between the target model and candidate bases, and then applies XOR-based delta compression within each cluster.
- **Default:** since FM-Delta [74] lacks a planning component, we enhance it with Hugging Face `base_model` metadata to group models by family and apply delta compression within each group. To reflect real-world metadata availability, we randomly sample 25.8% of models in the ZipLLM trace to carry metadata; the remaining models fall back to ZipNN [39] for standalone compression.

Compression Codec Baselines. To isolate the effect of the delta codec, we compare TensorX and FM-Delta against alternative compressors applied to the same base-delta pairs:

- **BitX [94]:** XOR-based delta compression that serves as the foundation for TensorX.
- **FM-Delta [74] vanilla:** which lacks BF16 support and runs single-threaded at ~100 MB/s.
- **ZipNN [39]:** groups floating-point numbers by byte component for single-tensor compression. It does not perform cross-tensor deduplication or delta compression; we additionally enable tensor-level deduplication (TensorDedup) for a fair comparison.
- **zstd [19]:** a general-purpose lossless compressor widely used in storage systems, applied to raw deltas.
- **OpenZL [20]:** a high-throughput lossless compression library that aggregates multiple compression algorithms.

Setup. We conduct our experiments on an Amazon EC2 [3] `c6a.48xlarge` instance, equipped with a 96-core (192-thread) AMD EPYC 7R13 processor and 384 GB of DRAM. Reduction ratios are computed from stored byte counts; the throughputs in Table 3 keep all data in DRAM; and Table 4 uses two real backends—Amazon S3 [2] in the same region, and an NVMe-attached EBS gp3 volume provisioned to 2,000 MiB/s, read cold and warm. Table 2 quantifies TensorDex’s metadata overhead.

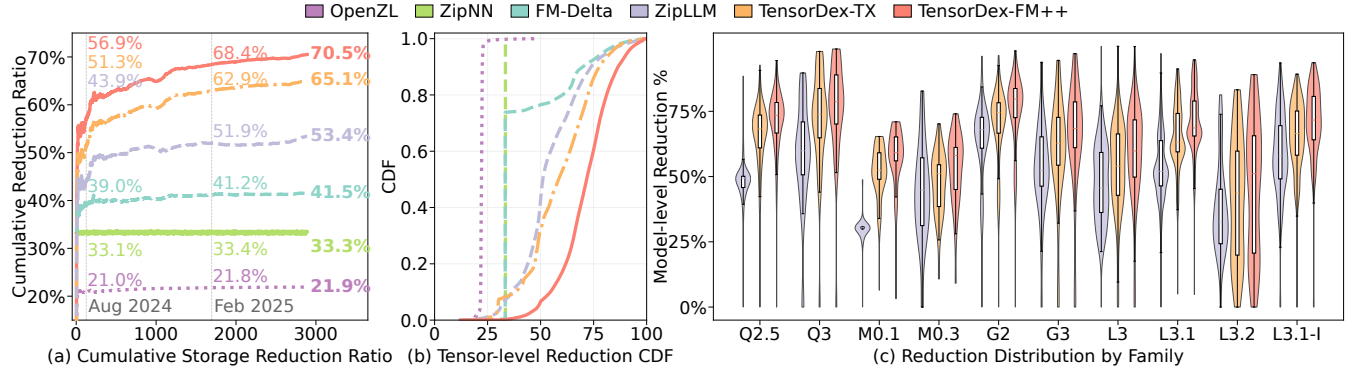


Figure 11. (a) Cumulative data reduction ratio as models are ingested into the ZipLLM-Trace corpus (ordered by creation time). (b) Per-tensor data reduction ratio CDF. (c) Per-model reduction ratio distributions by model family (Q: Qwen, M: Mistral, L: Llama, G: Gemma, I: Instruct). TensorDex-FM++ consistently achieves the highest median reduction across all ten families, followed by TensorDex-TX.

6.2 End-to-end Comparison

6.2.1 Data Reduction Ratio. To assess the end-to-end effectiveness of TensorDex, we run both TensorDex-TX and TensorDex-FM++ over the ZipLLM-Trace dataset. To mimic real-world usage on model hubs such as Hugging Face—where models arrive continuously—we incrementally ingest models in creation order and record the cumulative data reduction ratio. Fig. 11(a) reports how storage savings evolve as the corpus grows.

Both TensorDex variants consistently outperform all baselines and continue improving as more models are ingested, with no signs of saturation. TensorDex-FM++ achieves the highest reduction ratio (70.5%) owing to FM-Delta’s stronger compression, while TensorDex-TX reaches 65.1% with substantially higher throughput (§6.2.2). This sustained upward trend reflects TensorDex’s ability to discover new cross-tensor delta compression opportunities even at scale, whereas ZipLLM plateaus at 53.4% and standalone codecs—FM-Delta (37.1%), ZipNN (33.3%), and OpenZL (21.9%)—remain flat without cross-model pairing.

Both TensorDex variants remain robust because they do not rely on user-supplied model-card annotations.

Attributing the Gains. The comparisons above separate deduplication, planning, and the codec. *Deduplication is never double-counted:* TensorID first reduces 961,509 raw tensors (44.86 TB) to 741,689 unique ones (40.11 TB), and every ratio here—including the headline 70.5%—is measured against that post-deduplication baseline. *Fixing the codec isolates the planner:* TensorX is a byte-plane simplification of ZipLLM’s BitX at a comparable ratio, so ZipLLM and TensorDex-TX differ only in how pairs are chosen, and the 53.4% → 65.1% gap is FlexSplit’s. *Fixing the plan isolates the codec:* swapping TensorX for FM++ over the same pairs adds 5.4 points (65.1% → 70.5%) from its stronger entropy stage alone. Planning is a one-time ingestion cost, absent from the retrieval path, and can be batched offline.

To better understand the source of these gains, we examine the per-tensor reduction distribution (Fig. 11(b)). TensorDex-FM++ concentrates most tensors in the 60–90% reduction range, while standalone codecs (FM-Delta, ZipNN) cluster at fixed reduction levels (~37% and ~33%). The strong performance of TensorDex-FM++ is not solely due to the underlying FM-Delta codec. Rather, it is TensorDex’s tensor-centric planning—enabled by TensorSketch and FlexSplit—that uncovers high-quality cross-tensor pairings, allowing even a strong standalone compressor to operate on substantially lower-entropy deltas. In contrast, when applied without TensorDex’s planning, FM-Delta achieves only moderate, fixed compression gains. This demonstrates that the primary benefit comes from exposing previously hidden cross-tensor redundancy, with the compression codec further amplifying these gains.

Per Family Compression Breakdown. To understand how compression effectiveness varies across different model lineages, we break down the results by ten representative base families. Fig. 11(c) reports the model-level reduction ratio for all fine-tuned derivatives of each family.

Across families, TensorDex-FM++ consistently achieves the highest median reduction, followed by TensorDex-TX, with both substantially outperforming ZipLLM. The largest gaps appear on Qwen2.5, Mistral-v0.1, and Gemma-3, where ZipLLM’s reduction drops below 50% while TensorDex-FM++ stays above 65%—a 20-25 percentage-point advantage driven by TensorDex’s tensor-level matching, which discovers better bases even when fine-tuned variants drift from the nominal base model. For families like Qwen3 and Gemma-2, some individual models approach near-100% reduction, reflecting highly homogeneous derivatives where all methods perform well; yet even here the TensorDex variants’ wider violin “bellies” concentrate at higher ratios (75%+), indicating more consistent compression across the population.

Overall, this per-family analysis demonstrates that TensorDex is model-agnostic and robust regardless of the codec

Table 3. Data ingestion and retrieval throughput. Ingestion is end-to-end over the full pipeline (deduplication, fingerprinting, and codec encoding). All systems run with identical parallelism (192 threads), with all data in memory.

Method	Ingestion (MB/s)	Retrieval (MB/s)
ZipNN	1,473	9,575
ZipLLM	5,942	7,981
OpenZL	753	19,068
TensorDex-FM++	9,821	8,494
TensorDex-TX	22,916	28,393

choice, providing large benefits when lineage structure is complex while still improving upon ZipLLM even for tightly clustered families.

6.2.2 Throughput Performance. Table 3 reports end-to-end ingestion (deduplication + compression) and retrieval (decompression) throughput with 192 threads.

Data Ingestion Throughput. TensorDex’s ingestion pipeline consists of three stages—xxHash128 deduplication, TensorSketch fingerprinting, and codec encoding—all implemented as SIMD-accelerated kernels over zero-copy mmap buffers (§5.2). Because every stage is embarrassingly parallel across tensors and free of dynamic allocation, TensorDex scales linearly with thread count. TensorDex-TX sustains 22,916 MB/s aggregate ingestion throughput thanks to TensorX’s lightweight byte-plane pipeline. TensorDex-FM++ trades throughput for higher compression, while still remaining substantially faster than ZipLLM.

Data Retrieval Throughput. Retrieval cost is dominated by decompression, which is incurred on every model download. TensorDex-TX’s TensorX decompresses each chunk independently and recombines byte planes in parallel, sustaining 28,393 MB/s—well above typical network and storage bandwidth, ensuring that decompression is never the retrieval bottleneck. TensorDex-FM++ achieves 8,494 MB/s, balancing decompression speed with its higher compression effectiveness. ZipLLM and ZipNN achieve 7,981 MB/s and 9,575 MB/s, respectively.

Retrieval under Real Storage I/O. We repeat the same retrieval with blobs fetched from real backends—Amazon S3 with parallel ranged GETs, and an NVMe-attached EBS gp3 volume read cold and warm—obtaining 1.96–4.26 GB/s (Table 4), 3–4.5× below the same decoder running on DRAM-resident blobs. Retrieval is therefore I/O-bound on both tiers, confirming that decompression is never the bottleneck. Reconstruction is one hop: each delta references exactly one base, so restoring a tensor reads only that base plus its delta, and read amplification stays at 1.2–1.4× (median 1.27× across the corpus). Caching popular bases can reduce it further.

6.3 TensorSketch and Prediction Model

Similarity Search Baselines. To evaluate TensorSketch’s effectiveness in identifying compressible tensor pairs, we

Table 4. End-to-end retrieval under real storage I/O, using the same model and TensorDex-TX decoder as Table 3 and differing only in where the bytes come from. The *uncompressed pull* rows are a no-delta bandwidth reference over the model’s base blobs only.

Backend	Configuration	GB/s
Codec kernel only (DRAM)		28.6
Full decode path (DRAM)		7.9–8.8
Object store (S3; 64 in flight)	uncompressed pull	3.43
	cold, single-hop dedup	2.83
	warm bases	4.10
Block device (EBS gp3, 2000 MiB/s; 16 in flight)	cold, uncompressed pull	1.70
	cold, single-hop dedup	1.96
	cold, bases resident	2.92
	page-cache warm	4.26

compare against alternative similarity search methods paired with the same downstream planner:

- **Bit Distance [94]:** computes the exact pairwise Hamming distance over raw tensor bytes via brute-force comparison.
- **Raw Tensor HNSW [62]:** builds an HNSW index directly over raw tensor bytes, treating each full tensor as a high-dimensional vector for nearest-neighbor search.
- **LSH [34]:** applies locality-sensitive hashing to raw tensor representations, projecting tensors into hash buckets so that similar tensors are likely to collide.
- **RandProj 2048d [50, 89]:** applies a random linear projection (Johnson–Lindenstrauss) to reduce each tensor to a 2048-dimensional embedding, then builds an HNSW index over the projected vectors.

Sketch Method Comparison. We compare TensorSketch (backed by HNSW, see §5.1) against Raw Tensor HNSW, LSH, and RandProj 2048d across three major model families (Qwen, Llama, Gemma). As shown in Fig. 12, TensorSketch dominates on all three metrics.

On *Recall@1*, TensorSketch achieves near-perfect recall (~1.00) across all families, while Raw Tensor HNSW and LSH range from 0.60 to 0.77 and RandProj 2048d from 0.32 to 0.41. On *greedy reduction ratio*, TensorSketch consistently achieves the highest reduction, reaching approximately 60% on the Gemma family, demonstrating that its high recall directly translates to better compression planning. On *end-to-end QPS*, TensorSketch sustains over 25,000 QPS—500–800× faster than Raw Tensor HNSW (~31–45 QPS)—because its compact sketch reduces the per-query data footprint from ~35 MB (Raw Tensor HNSW) to only ~8 KB, a >4,000× reduction in I/O.

The two approximations in this path—sketching, then HNSW—compound without cost, because all that matters for greedy attach is that the top-1 candidate stays stable. TensorSketch matches exact bit distance at Recall@1 = 1.00 (Fig. 12(a)), and HNSW over those same sketches reaches Recall@1 = 0.9986 against exact search, yielding an *identical* FlexSplit outcome (65.1% reduction, 6,482 bases) while

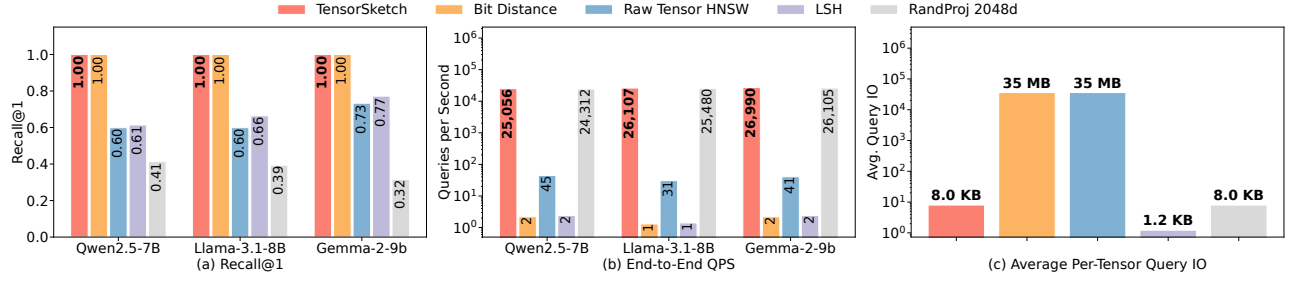


Figure 12. Performance Comparison of TensorSketch. Comparison of TensorSketch against baselines across Qwen, Llama, and Gemma families. (a) **Recall@1** (top-1 match accuracy): TensorSketch maintains a perfect 1.00 Recall@1, matching the exact Bit Distance baseline. (b) **End-to-End QPS** (queries per second): TensorSketch achieves over 25,000 QPS, representing a 4-order-of-magnitude speedup (up to 20,082x) compared to Bit Distance. (c) **Average Query IO**: TensorSketch reduces the per-tensor query I/O from 35 MB (Bit Distance) to 8.0 KB, achieving a 4,517x reduction in IO overhead while maintaining significantly higher accuracy than other low-IO baselines like LSH.

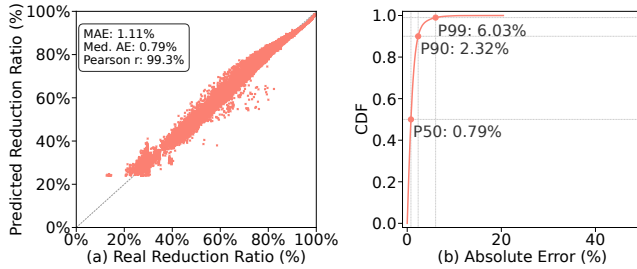


Figure 13. Validation of the reduction ratio prediction model. The coefficients ($\alpha, \beta, \gamma, \epsilon$) come from a single global fit over 1M training pairs; the figure evaluates them on 5.7M held-out validation pairs. §6.3 additionally reports a cross-family holdout. (Left) Predicted vs. real reduction ratio. (Right) CDF of absolute prediction error.

clustering 3.8x faster. Sweeping efSearch from 16 to 256 changes nothing.

Prediction Model Accuracy. Next, we evaluate the accuracy of the reduction prediction model. Fig. 13(a) plots predicted vs. real reduction ratio for 5.7M tensor pairs. The scatter tightly follows the diagonal (Pearson $r = 0.993$), with a mean absolute error of only 1.11% and a median absolute error of 0.79%. The close alignment across the full range—from 20% to near-perfect reduction—shows that the lightweight sketch-based predictor faithfully captures delta compressibility without requiring full-tensor compression.

Fig. 13(b) presents the CDF of absolute prediction error. The model achieves a median error (P50) of 0.79%, meaning that half of all predictions differ from ground truth by less than one percentage point. Even at the tail, errors remain small: P90 = 2.32% and P99 = 6.03%. This level of accuracy is more than adequate for our use case: planning and clustering depend primarily on relative ordering among candidate bases rather than exact ratio magnitudes. The prediction model thus provides a lightweight yet effective mechanism for large-scale storage optimization.

To rule out family leakage, we refit on one set of families and test on disjoint ones: trained on Qwen+Llama (451K

pairs) and evaluated on never-seen Gemma+Mistral (83K pairs), MAE rises only from 0.0230 to 0.0265 and Pearson r is unchanged (0.978 vs. 0.977). The predictor’s inputs are $\hat{p}$ and its binary entropy—numerical properties independent of model family—so one global regression fits all families.

6.4 FlexSplit Analysis

Evaluating Solvers. To assess the scalability and quality of FlexSplit’s heuristic base selection, we compare against exact and classical approximation solvers:

- **ILP (Gurobi) [37]:** an integer linear programming formulation solved by Gurobi, providing the global optimum but intractable beyond a few hundred tensors.
- **Primal-Dual [47]:** the classical 3-approximation algorithm using the primal-dual schema. It offers provable guarantees but requires the full pairwise cost matrix and assumes static inputs.

Scalability. We benchmark the end-to-end solving time of FlexSplit against ILP and Primal-Dual over up to 800 Qwen2.5-7B tensors using two representative tensor types (q_proj_weight and v_proj_weight). The results highlight a sharp contrast in scalability (Fig. 14). As the number of tensors grows from 100 to 800, ILP’s solving time increases super-linearly (exceeding 70 s at 800 tensors), making it impractical for production-scale model hubs. Primal-Dual grows linearly but still increases with scale. In contrast, FlexSplit maintains *near-constant* solving time across all scales, approaching zero even at 800 tensors. Its fingerprint-guided assignments and lightweight predictive splitting require only sketch-level distance evaluations, avoiding the expensive global cost computation that dominates the baselines.

Despite this vastly lower solve time, FlexSplit achieves a *near-optimal* reduction ratio—stable at 0.60-0.75 and consistently within a few percentage points of the ILP optimum—demonstrating that the algorithm is both compute-efficient and high quality. This scalability enables TensorDex to continuously ingest large model families while retaining ILP-level compression effectiveness.

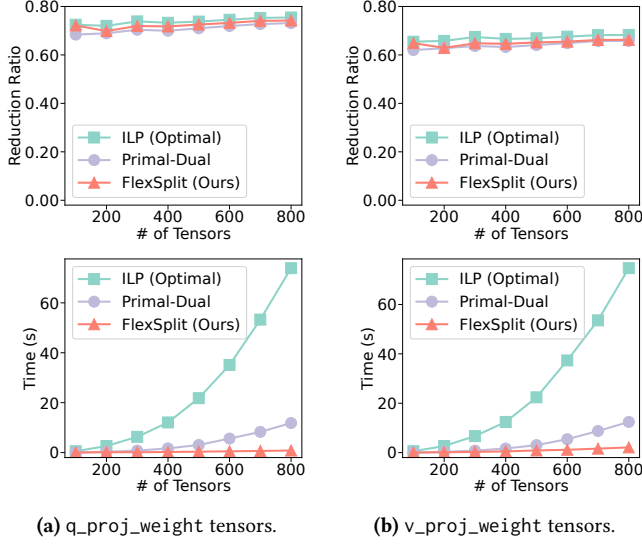


Figure 14. Scalability of FlexSplit vs. ILP and Primal-Dual solvers on two representative tensor types. **(Top)** Reduction ratio remains near-optimal for FlexSplit across all scales. **(Bottom)** Solving time: ILP grows super-linearly and Primal-Dual grows linearly, while FlexSplit maintains near-constant time.

Splitting Behavior. We examine how FlexSplit’s two-phase structure behaves in practice over the full ZipLLM-Trace corpus. Fig. 15 characterizes the clusters produced by Phase I greedy assignment. The left panel shows cluster size distributions on a log scale: non-split clusters are concentrated at small sizes (<10 tensors), while split-targeted clusters (i.e., clusters that will trigger a split in Phase II) dominate the 10^2 – 10^3 range, aggregating heterogeneous tensors from diverse fine-tuning lineages. The right panel reveals a corresponding compressibility gap: split-targeted clusters peak at low reduction ratios (0.25–0.50), whereas non-split clusters peak near 0.75. This confirms that Phase II applies a dual criterion—it triggers a split only when a cluster is both sufficiently large and sufficiently inefficient to warrant refinement. Overall, 58.9% of clusters require no split at all, confirming that FlexSplit’s refinement is *surgical* rather than pervasive.

Fig. 16 quantifies the improvement from Phase II splitting over 1,352 clusters. The left panel plots per-cluster reduction ratio before (greedy assignment) vs. after (FlexSplit): 98.8% of split clusters improve, with the median ratio increasing from 0.469 to 0.650. Only 1.2% of cases show marginal regression, concentrated near zero. The right panel shows the distribution of net gain: most improvements fall between 0.1 and 0.3, meaning Phase II typically adds 10–30% additional reduction per cluster. This demonstrates that Phase II reliably reassigns outlier tensors—those with poor fit to their initial base—to better-matched centers, directly translating into improved storage efficiency without accessing full tensor contents.

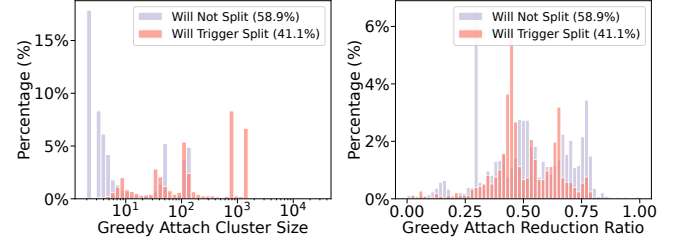


Figure 15. Cluster characteristics after Phase I greedy assignment, categorized by whether Phase II triggers a split. **(Left)** Cluster size distribution (log-scale). **(Right)** Reduction ratio distribution.

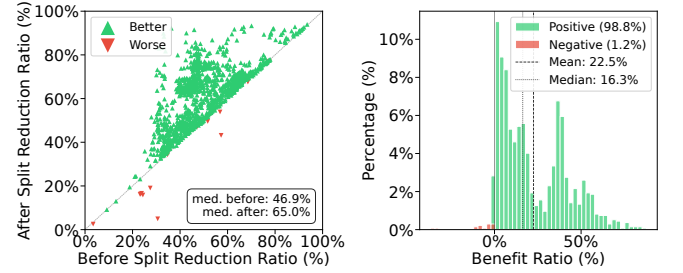


Figure 16. Effect of Phase II FlexSplit splitting on per-cluster reduction ratio ($n = 1,352$ clusters). **(Left)** Scatter of greedy assignment (Phase I) vs. FlexSplit (Phase II) reduction ratio. **(Right)** Distribution of net gain (FlexSplit minus greedy).

7 Conclusion

This paper presents TensorDex, a compact, tensor-centric AI model storage system. TensorDex addresses the granularity mismatch in existing delta compression approaches by exploiting data-driven cross-tensor relationships. TensorDex synthesizes four novel techniques: a compression-aware fingerprint that captures bit-level tensor similarity, a lightweight prediction model that estimates delta compressibility without accessing full data, an adaptive clustering planner that discovers high-quality base-delta tensor pairings at scale, and two pluggable high-performance delta codecs. Together, these techniques enable an efficient, metadata-oblivious compression pipeline that substantially reduces storage while maintaining high performance in large-scale real-world model repositories.

Acknowledgment

We thank our shepherd and the anonymous reviewers for their valuable feedback and comments, which improved the paper. This research was supported in part by NSF grants CNS-2322860, OAC-2411009, and OAC-2403313. We also acknowledge support from NSF CloudBank for providing AWS credits, thank Adobe for their generous research gift, and acknowledge support from the Commonwealth Cyber Initiative (CCI), an investment in the advancement of cyber research, innovation, and workforce development. For more information about CCI, visit cyberinitiative.org.

References

- [1] Jyrki Alakuijala, Andrea Farruggia, Paolo Ferragina, Evgenii Kliuchnikov, Robert Obryk, Zoltan Szabadka, and Lode Vandevenne. Brotli: A general-purpose data compressor. *ACM Transactions on Information Systems*, 2019.
- [2] Amazon Web Services. Amazon S3: A Simple Storage Service. <https://aws.amazon.com/s3/>, 2006.
- [3] Amazon Web Services. Amazon ec2 - elastic compute cloud. <https://aws.amazon.com/ec2/>, 2026. Accessed: 2026-04-02.
- [4] Hyung-Chan An, Ashkan Norouzi-Fard, and Ola Svensson. Dynamic facility location via exponential clocks. 13(2), February 2017.
- [5] Alexandr Andoni and Ilya Razenshteyn. Optimal data-dependent hashing for approximate near neighbors. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '15, page 793–801, New York, NY, USA, 2015. Association for Computing Machinery.
- [6] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: high-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.*, 9(4):288–299, December 2015.
- [7] Vijay Arya, Naveen Garg, Rohit Khandekar, Adam Meyerson, Kamesh Munagala, and Vinayaka Pandit. Local search heuristics for k -median and facility location problems. *SIAM Journal on Computing*, 33(3):544–562, 2004.
- [8] J.E. Beasley. Lagrangean heuristics for location problems. *European Journal of Operational Research*, 65(3):383–399, 1993.
- [9] Ksenia Bestuzheva, Mathieu Besançon, Wei-Kun Chen, Antonia Chmiela, Tim Donkiewicz, Jasper van Doornmalen, Leon Eifler, Oliver Gaul, Gerald Gamrath, Ambros Gleixner, et al. The SCIP Optimization Suite 8.0. Technical Report, Optimization Online, 2021.
- [10] Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. When is “nearest neighbor” meaningful? In *Proceedings of the 7th International Conference on Database Theory (ICDT)*, pages 217–235, 1999.
- [11] Alina Beygelzimer, Sham Kakade, and John Langford. Cover trees for nearest neighbor. In *Proceedings of the 23rd International Conference on Machine Learning*, ICML '06, page 97–104, New York, NY, USA, 2006. Association for Computing Machinery.
- [12] Kushal Raj Bhandari, Pin-Yu Chen, and Jianxi Gao. Forecasting open-weight ai model growth on huggingface, 2025.
- [13] Davis Blalock, Samuel Madden, and John Guttag. Sprintz: Time series compression for the internet of things. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 2(3):1–23, 2018.
- [14] Martin Burtscher and Paruj Ratanaworabhan. High throughput compression of double-precision floating-point data. In *2007 Data Compression Conference (DCC'07)*, pages 293–302. IEEE, 2007.
- [15] Moses Charikar, Kevin Chen, and Martin Farach-Colton. Finding frequent items in data streams. In *International Colloquium on Automata, Languages, and Programming*, pages 693–703. Springer, 2002.
- [16] Julien Chaumond. “We have doubled the amount of total storage on Hugging Face in the past five months. At this rate, we will cross 1 Exabyte before the end of the year.”. https://www.linkedin.com/posts/julienchaumond_we-have-doubled-the-amount-of-total-storage-share-7467552847988174849-HY5S/, 2026. Accessed: 2026-08-16.
- [17] Cloudflare. Cloudflare r2. <https://www.cloudflare.com/developer-platform/products/r2/>.
- [18] Yann Collet. xxhash - extremely fast hash algorithm. <https://github.com/Cyan4973/xxHash>, 2012.
- [19] Yann Collet and Murray Kucherawy. Zstandard compression and the application/zstd media type. Technical report, 2018.
- [20] Yann Collet, Nick Terrell, W. Felix Handte, Danielle Rozenblit, Victor Zhang, Kevin Zhang, Yaelle Goldschlag, Jennifer Lee, Elliot Gorokhovskiy, Yonatan Komornik, Daniel Riegel, Stan Angelov, and Nadav Rotem. Openz: A graph-based model for compression, 2025.
- [21] Xingyu Dang, Christina Baek, Kaiyue Wen, Zico Kolter, and Aditi Raghunathan. Weight ensembling improves reasoning in language models, 2025.
- [22] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the Twentieth Annual Symposium on Computational Geometry*, SCG '04, page 253–262, New York, NY, USA, 2004. Association for Computing Machinery.
- [23] davanstrien. Add ‘base_model’ metadata to the automatically generated model card. GitHub Issue #938, huggingface/peft, 2023. Accessed: 2025-12-12.
- [24] Dell Technologies. Understanding data domain compression. <https://www.dell.com/en-us/shop/storage-servers-and-networking-for-business/sf/powerprotect-data-domain>, 2023.
- [25] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefer, and Dan Alistarh. Spqr: A sparse-quantized representation for near-lossless llm weight compression. *arXiv preprint arXiv:2306.03078*, 2023.
- [26] Peter Deutsch. Deflate compressed data format specification version 1.3. Technical report, 1996.
- [27] Sheng Di and Franck Cappello. Fast error-bounded lossy hpc data compression with sz. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 730–739. IEEE, 2016.
- [28] James Diffenderfer, Alyson L Fox, Jeffrey A Hittinger, Geoffrey Sanders, and Peter G Lindstrom. Error analysis of zfp compression for floating-point data. *SIAM Journal on Scientific Computing*, 41(3):A1867–A1898, 2019.
- [29] Jarek Duda. Asymmetric numeral systems: entropy coding combining speed of huffman coding with compression rate of arithmetic coding. *arXiv preprint arXiv:1311.2540*, 2013.
- [30] Hugging Face. Hugging face. <https://huggingface.co/>, 2023.
- [31] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- [32] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow.*, 12(5):461–474, January 2019.
- [33] Junhao Gan, Jianlin Feng, Qiong Fang, and Wilfred Ng. Locality-sensitive hashing scheme based on dynamic collision counting. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, SIGMOD '12, page 541–552, New York, NY, USA, 2012. Association for Computing Machinery.
- [34] Aristides Gionis, Piotr Indyk, Rajeev Motwani, et al. Similarity search in high dimensions via hashing. In *Vldb*, volume 99, pages 518–529, 1999.
- [35] GitHub. Git large file storage (lfs). <https://github.com/git-lfs/git-lfs>, 2024.
- [36] Almog Gueta, Elad Venezian, Colin Raffel, Noam Slonim, Yoav Katz, and Leshem Choshen. Knowledge is a region in weight space for fine-tuned language models. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [37] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2026.
- [38] R. W. Hamming. Error detecting and error correcting codes. *The Bell System Technical Journal*, 29(2):147–160, 1950.
- [39] Moshik Hershcovitch, Andrew Wood, Leshem Choshen, Guy Gironmsky, Roy Leibovitz, Ilias Ennmouri, Michal Malka, Peter Chin, Swaminathan Sundararaman, and Danny Harnik. Zipnn: Lossless compression for ai models. *arXiv preprint arXiv:2411.05239*, 2024.

- [40] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- [41] David A Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.
- [42] Hugging Face. Model cards - hugging face documentation. <https://huggingface.co/docs/hub/en/model-cards>, 2024.
- [43] Hugging Face. Safetensors documentation. <https://huggingface.co/docs/safetensors/en/index>, 2024.
- [44] Hugging Face. Ready, xet, go! a new era of dataset versioning. <https://huggingface.co/spaces/jsulz/ready-xet-go>, 2025. Accessed: 2025-11-09.
- [45] IBM. IBM ILOG CPLEX Optimization Studio, 2022.
- [46] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*, pages 604–613, 1998.
- [47] Kamal Jain and Vijay V. Vazirani. Approximation algorithms for metric facility location and k -median problems using the primal-dual schema and Lagrangian relaxation. *Journal of the ACM*, 48(2):274–296, 2001.
- [48] Dong-Hwan Jang, Sangdoo Yun, and Dongyoon Han. Model stock: All we need is just a few fine-tuned models. In *Computer Vision – ECCV 2024: 18th European Conference, Milan, Italy, September 29–October 4, 2024, Proceedings, Part XLIV*, page 207–223, Berlin, Heidelberg, 2024. Springer-Verlag.
- [49] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7b. <https://arxiv.org/abs/2310.06825>, 2023.
- [50] William B Johnson, Joram Lindenstrauss, et al. Extensions of lipschitz mappings into a hilbert space. *Contemporary mathematics*, 26(189-206):1, 1984.
- [51] Herve J  gou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.
- [52] Maximilian Kuschewski, David Sauerwein, Adnan Alhomssi, and Viktor Leis. Btrblocks: Efficient columnar compression for data lakes. *Proceedings of the ACM on Management of Data*, 1(2):1–26, 2023.
- [53] Benjamin Laufer, Hamidah Oderinwale, and Jon Kleinberg. Anatomy of a machine learning ecosystem: 2 million models on hugging face, 2025.
- [54] Panagiotis Liakos, Katia Papakonstantinou, and Yannis Kotidis. Chimp: efficient lossless floating point compression for time series databases. *Proceedings of the VLDB Endowment*, 15(11):3058–3070, 2022.
- [55] Weixin Liang, Nazneen Rajani, Xinyu Yang, Ezinwanne Ozoani, Eric Wu, Yiqun Chen, Daniel Scott Smith, and James Zou. What’s documented in ai? systematic analysis of 32k ai model cards. *arXiv preprint arXiv:2402.05160*, 2024.
- [56] Xin Liang, Sheng Di, Dingwen Tao, Zizhong Chen, and Franck Cappello. An efficient transformation scheme for lossy data compression with point-wise relative error bound. In *2018 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 179–189. IEEE, 2018.
- [57] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100, 2024.
- [58] Peter Lindstrom. Fixed-rate compressed floating-point arrays. *IEEE transactions on visualization and computer graphics*, 20(12):2674–2683, 2014.
- [59] Chunwei Liu, Hao Jiang, John Paparrizos, and Aaron J Elmore. Decomposed bounded floats for fast compression and queries. *Proceedings of the VLDB Endowment*, 14(11):2586–2598, 2021.
- [60] Kejing Lu, Mineichi Kudo, Chuan Xiao, and Yoshiharu Ishikawa. Hvs: hierarchical graph structure based on voronoi diagrams for solving approximate nearest neighbor search. *Proc. VLDB Endow.*, 15(2):246–258, October 2021.
- [61] Josh MacDonald. *File system support for delta compression*. PhD thesis, Masters thesis. Department of Electrical Engineering and Computer Science ..., 2000.
- [62] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. Mach. Intell.*, 42(4):824–836, April 2020.
- [63] Yuri Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems*, 45:61–68, 2014.
- [64] Detlev Marpe, Heiko Schwarz, and Thomas Wiegand. Context-based adaptive binary arithmetic coding in the h. 264/avc video compression standard. *IEEE Transactions on circuits and systems for video technology*, 13(7):620–636, 2003.
- [65] Meta AI. Introducing llama 3.1: Our most capable models to date. <https://ai.meta.com/blog/meta-llama-3-1/>, 2024.
- [66] Meta AI. Introducing meta llama 3: The most capable openly available llm to date. <https://ai.meta.com/blog/meta-llama-3/>, 2024.
- [67] Meta AI. Llama 3.1 8b. <https://huggingface.co/meta-llama/Llama-3.1-8B>, 2024.
- [68] Meta AI. Llama 3.2: Revolutionizing edge ai and vision with open, customizable models. <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>, 2024.
- [69] Meta AI. The llama 4 herd: The beginning of a new era of natively multimodal ai innovation. <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, 2025.
- [70] Jason Mohoney, Devesh Sarda, Mengze Tang, Shihabur Rahman Chowdhury, Anil Pacaci, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. Quake: Adaptive indexing for vector search. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 153–169, 2025.
- [71] Athicha Muthitacharoen, Benjie Chen, and David Mazieres. A low-bandwidth network file system. In *Proceedings of the eighteenth ACM symposium on Operating systems principles*, pages 174–187, 2001.
- [72] NetApp. Netapp ontap 9 storage efficiency guide. Technical Report TR-3966, NetApp, 2020.
- [73] NetApp. Ontap data management software. <https://www.netapp.com/data-management/ontap-data-management-software/>, 2024.
- [74] Wanyi Ning, Jingyu Wang, Qi Qi, Mengde Zhu, Haifeng Sun, Daixuan Cheng, Jianxin Liao, and Ce Zhang. Fm-delta: Lossless compression for storing massive fine-tuned foundation models. *Advances in Neural Information Processing Systems*, 37:66796–66825, 2024.
- [75] Myoungwon Oh, Sungmin Lee, Samuel Just, Young Jin Yu, Duck-Ho Bae, Sage Weil, Sangyeun Cho, and Heon Y. Yeom. TiDedup: A new distributed deduplication architecture for ceph. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 855–869, 2023.
- [76] William B. Pennebaker, Joan L. Mitchell, Glen G Langdon, and Ronald B Arps. An overview of the basic principles of the q-coder adaptive binary arithmetic coder. *IBM Journal of research and development*, 32(6):717–726, 1988.
- [77] PostgreSQL. PostgreSQL: The World’s Most Advanced Open Source Relational Database. <https://www.postgresql.org/>.

- [78] Morgane Riviere et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.
- [79] Raunak Shah, Zhaoheng Li, and Yongjoo Park. Qstore: Quantization-aware compressed model storage. *Proc. VLDB Endow.*, 19(3):388–398, March 2026.
- [80] Gregory Shakhnarovich, Trevor Darrell, and Piotr Indyk. *Nearest-Neighbor Methods in Learning and Vision: Theory and Practice (Neural Information Processing)*. The MIT Press, 2006.
- [81] Chanop Silpa-Anan and Richard Hartley. Optimised kd-trees for fast image descriptor matching. In *2008 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1–8, 2008.
- [82] Michal Stabno and Robert Wrembel. RLh: Bitmap compression technique based on run-length and huffman encoding. In *Proceedings of the ACM tenth international workshop on Data warehousing and OLAP*, pages 41–48, 2007.
- [83] Trevor Stalaker, Nathan Wintersgill, Oscar Chaparro, Laura A. Heymann, Massimiliano Di Penta, Daniel M German, and Denys Poshyvanyk. An empirical analysis of machine learning model and dataset documentation, supply chain, and licensing challenges on hugging face. *ACM Trans. Softw. Eng. Methodol.*, November 2025. Just Accepted.
- [84] Zhaoyuan Su, Ammar Ahmed, Zirui Wang, Ali Anwar, and Yue Cheng. Everything you always wanted to know about storage compressibility of pre-trained ml models but were afraid to ask. *Proc. VLDB Endow.*, 17(8):2036–2049, April 2024.
- [85] Muhammad Asif Suryani, Saurav Karmakar, Brigitte Mathiak, and Philipp Mayr. Model card metadata collection from hugging face to foster multidisciplinary ai research: A dataset. In *Proceedings of the 14th International Conference on Data Science, Technology and Applications*, pages 583–590, 2025.
- [86] Dingwen Tao, Sheng Di, Zizhong Chen, and Franck Cappello. Significantly improving lossy compression for scientific data sets based on multidimensional prediction and error-controlled quantization. In *2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 1129–1139. IEEE, 2017.
- [87] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.
- [88] Andrew Tridgell. Efficient algorithms for sorting and synchronization. 1999.
- [89] Santosh S Vempala. *The random projection method*, volume 65. American Mathematical Soc., 2005.
- [90] Jeffrey Scott Vitter. Design and analysis of dynamic huffman codes. *Journal of the ACM (JACM)*, 34(4):825–845, 1987.
- [91] Deepak Vohra. Practical hadoop ecosystem. *Chapter in Apache Parquet*, 177:178, 2016.
- [92] Borui Wan, Mingji Han, Yiyao Sheng, Yanghua Peng, Haibin Lin, Mofan Zhang, Zhichao Lai, Menghan Yu, Junda Zhang, Zuquan Song, Xin Liu, and Chuan Wu. ByteCheckpoint: A unified checkpointing system for large foundation model development. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 559–578, Philadelphia, PA, April 2025. USENIX Association.
- [93] Jingdong Wang and Shipeng Li. Query-driven iterated neighborhood graph search for large scale indexing. In *Proceedings of the 20th ACM International Conference on Multimedia, MM '12*, page 179–188, New York, NY, USA, 2012. Association for Computing Machinery.
- [94] Zirui Wang, Tingfeng Lan, Zhaoyuan Su, Juncheng Yang, and Yue Cheng. Ziplm: Efficient llm storage via model-aware synergistic data deduplication and compression. *arXiv preprint arXiv:2505.06252*, 2025.
- [95] Welch. A technique for high-performance data compression. *Computer*, 17(6):8–19, 1984.
- [96] XetHub. From Files to Chunks: Improving HF Storage Efficiency. <https://huggingface.co/blog/from-files-to-chunks>, 2024.
- [97] Wen Xia, Yukun Zhou, Hong Jiang, Dan Feng, Yu Hua, Yuchong Hu, Qing Liu, and Yucheng Zhang. {FastCDC}: A fast and efficient {Content-Defined} chunking approach for data deduplication. In *2016 USENIX Annual Technical Conference (USENIX ATC 16)*, pages 101–114, 2016.
- [98] Siqi Xiang, Sheng Wang, Xiaokui Xiao, Cong Yue, Zhanhao Zhao, and Beng Chin Ooi. Neurstore: Efficient in-database deep learning model management system. *Proc. ACM Manag. Data*, 3(6), December 2025.
- [99] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099. PMLR, 2023.
- [100] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, Peng Cheng, and Mao Yang. SPFresh: Incremental in-place update for billion-scale vector search. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pages 545–561, 2023.
- [101] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [102] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [103] Xiaozhe Yao, Qinghao Hu, and Ana Klimovic. Deltazip: Efficient serving of multiple full-model-tuned llms. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 110–127, 2025.
- [104] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *Advances in Neural Information Processing Systems*, 35:27168–27183, 2022.
- [105] Ofir Zafrir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. Q8bert: Quantized 8bit bert. In *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing-NeurIPS Edition (EMC2-NIPS)*, pages 36–39. IEEE, 2019.
- [106] Jacob Ziv and Abraham Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on information theory*, 23(3):337–343, 1977.